



Distributed and Parallel Programming course, University of Camerino

PARALLEL PROGRAMMING

IMPLEMENTATION OF A SIMPLIFIED SOLUTION TO THE
N-BODY PROBLEM



Distributed and Parallel Programming course

The N-body problem is a classic issue in physics and numerical simulation, which involves **calculating the motion of N bodies that interact with each other through forces**, typically gravitational ones.

In order to make the problem computationally tractable several simplifications were introduced.

- The bodies are modeled as **point masses**
- The motion is restricted to **two dimensions**
- The instantaneous **variability** of **acceleration**
- The **conic trajectories** of the bodies

The primary objective of this work is to develop a simplified simulation of the N-body problem through a parallel implementation in C

The Algorithm

- Define a **Body structure**: mass, position and speed coordinates.
- **Initialize bodies** through **random** generation or by **reading** data from a file.
- The **master** process **distributes** the assigned portions of the bodies array to all processes

Each process is responsible for updating only the state of the bodies assigned to it.

- **Compute the state** considering **interactions** among its **local bodies**
 - **Send** its local bodies to the **next** node
 - **Receive** the bodies belonging to **previous** node
 - **Compute** the **increment of the state** considering the **bodies received**
- **Update the state** of each body adding the accumulated increment
- **Collect** the final state of all the bodies into the **master**.



Distributed and Parallel Programming course

Objectives

Analyze how **efficiency and speed-up** vary:

- 1) Increasing the processors with constant input
- 2) Increasing the processors and the input size of a factor 2

To assess whether **blocking or non-blocking** communication is the most performant

The **execution time** was analyzed.

Data were collected over several configurations using **1 up to 12 processors** and incrementing the input size from **10.000 to 100.000 bodies**.

The **input data were** not randomly generated, but were **read** from a file, data.txt.

To compute speed-up and efficiency, the results were not based on a single run, but rather on the **average of five runs** for each **configuration**.

Data Analysis

When the number of **processors was increased**, the **speed increase**.

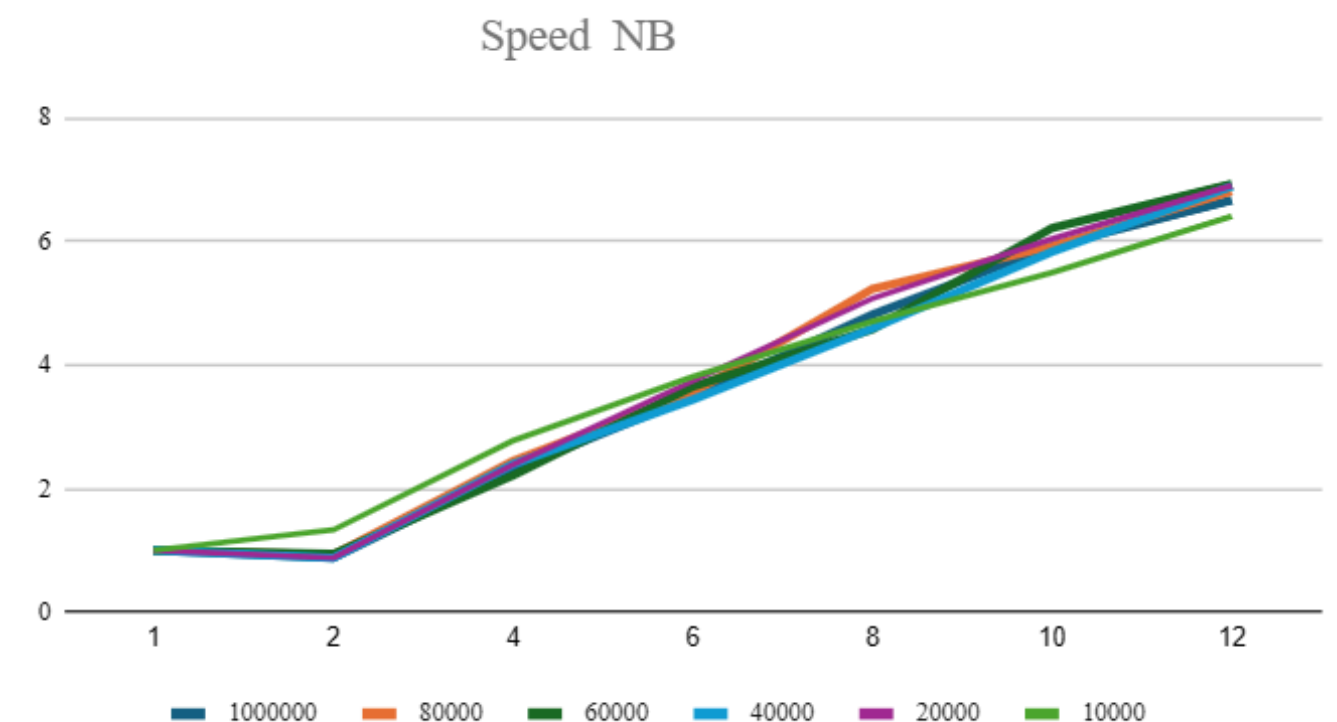
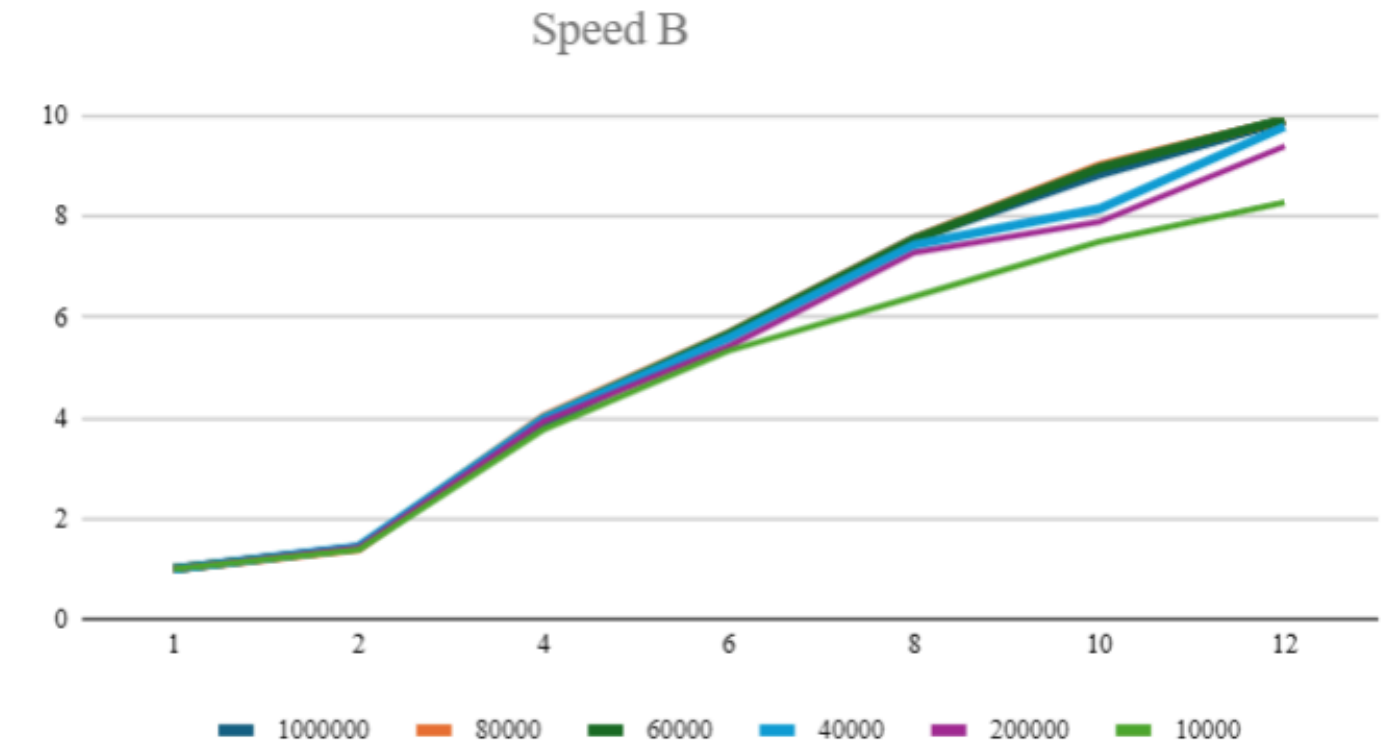
The speed is computed by dividing serial execution time and parallel execution time

$$S = \frac{T_{\text{serial}}}{T_{\text{parallel}}}$$

Then a comparison with the **ideal parallel time** was done and it was possible to point out different behaviour between programs.

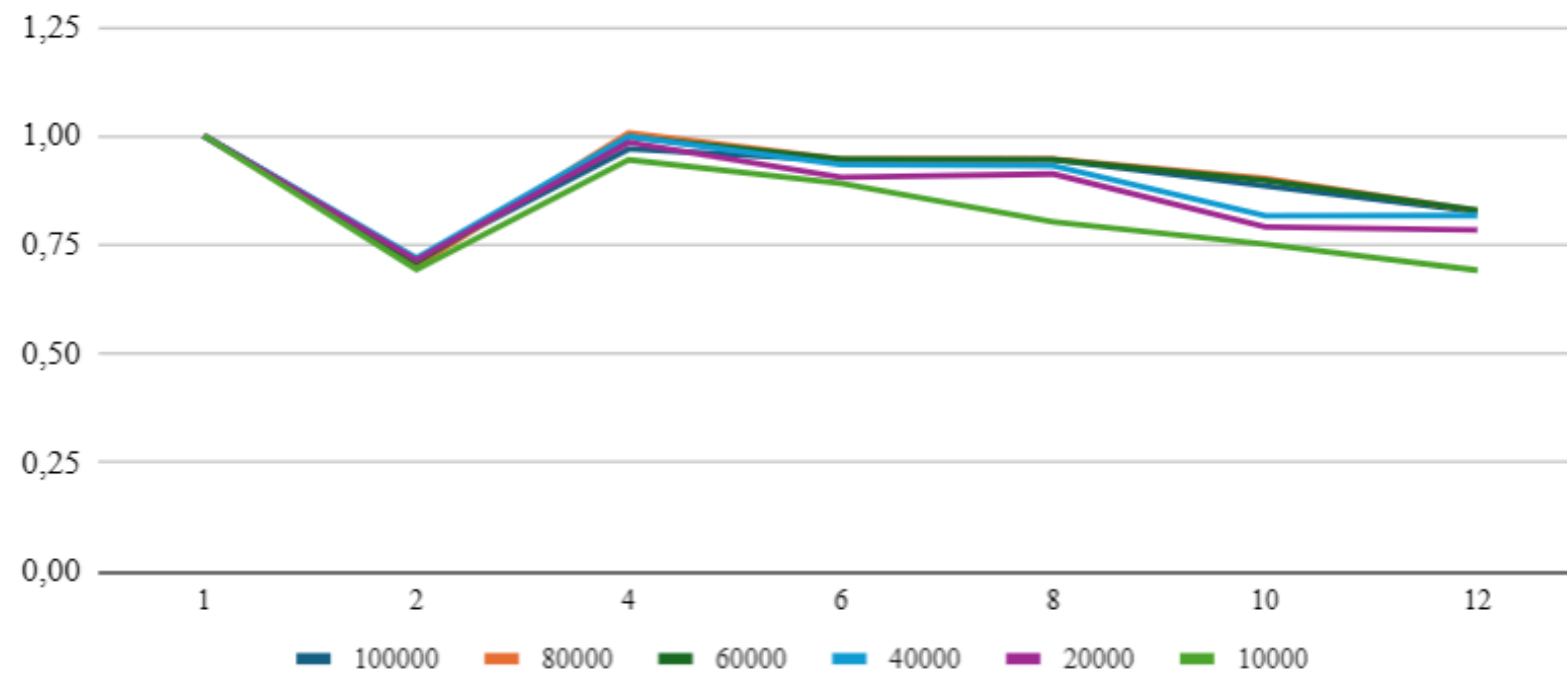
nBodyNB program demonstrates a **reduction in overhead** as the number of processors increases.

By contrast, the **blocking program** (excluding the two-processor case) exhibits an **upward trend** in overhead.



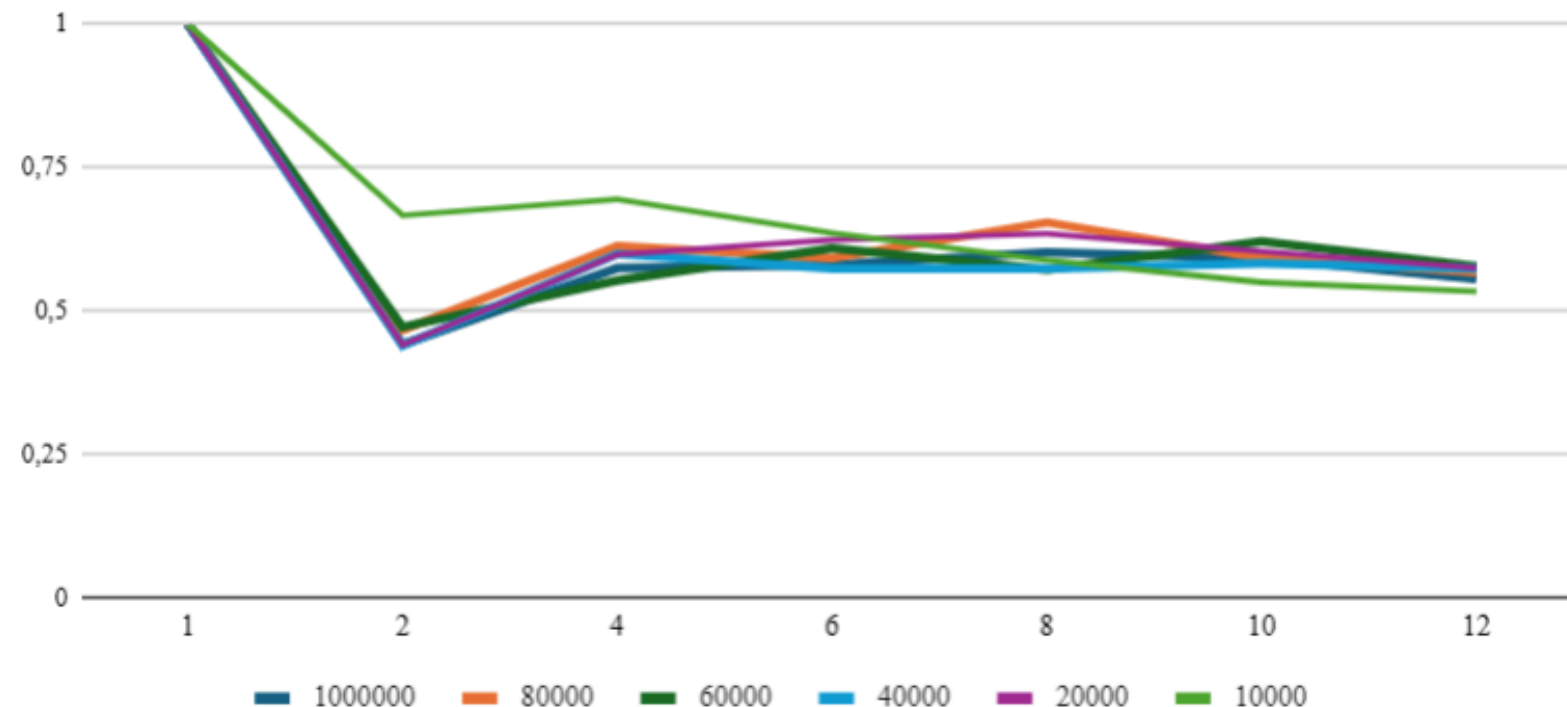
Distributed and Parallel Programming course

Efficiency B



As for efficiency, both programs show a decreasing trend, as expected from **Amdahl's Law**: as the number of processors increases, the marginal gain in performance decreases due to **non-parallelizable** sections of the code and **communication**.

Efficiency NB



Both programs seems to be scalable.

Weak scalability

If we can keep the **efficiency fixed** by **increasing processes** at the same rate as we **increase the problem size**, then the program is said to be weakly scalable.

Strong scalability

If we can keep the **efficiency fixed** by **increasing processes** without increasing the problem size, the program is said to be strongly scalable.



Distributed and Parallel Programming course

Hypothesis

It was expected that nBodyNB would outperform nBody, precisely because **non-blocking** communication should enable **better performance** by hiding communication latency.

Results

In this specific context, the advantage of non-blocking communication over blocking one, **never emerges**.

Explanation

The fact is that the **MPI Sendrecv** call is much faster because it is executed as a single call, almost as if it were an **atomic action**.

Even though it is a blocking operation, it is still faster than two non-blocking calls.

Additionally, as previously mentioned, **non-blocking** functions **require a wait call**.

Wait call is necessary to correctly compute the new state of the bodies, we must **ensure** that the **all the data have been received**.

It is important to emphasize that when the **wait is invoked** the program's behavior becomes **temporary blocking**.